

Chair of Computer Architecture (M. Sc.)

Model Enumeration using IPASIR-UP

Master's Thesis

First Examiner

Prof. Dr. Armin Biere

Second Examiner

Prof. Dr. xyz

Author

Timpe Horig

Matrikelnummer: 5108196

Geburtsort: Neustadt, Deutschland

Working Period

start date - end date

Abstract

Contents

1	Introduction	1
2	IPASIR-UP	2
3	Deciding	5
4	Chronological Backtracking	6
4.1	Allowing force_backtrack in cb_check_found_model	6
4.2	(i) Backtracking through book keeping	6
4.2.1	Backtracking after a model was found	6
4.2.2	Conflict induced backtracking	7
4.3	(ii) Backtracking through propagation	7
5	Implicant Shrinking	7
6	Extending IPASIR-UP	8
6.0.1	Making IPASIR-UP the only interface needed	8
6.0.2	Implicant Shrinking	8
6.0.3	Optimization	8
7	Results	8
8	Conclusion	8

1 Introduction

The Boolean satisfiability problem (SAT) is defined as follows:

Given a propositional formula F over variables $V = \{v_1, \dots, v_n\}$, decide whether there exists an assignment $\alpha : V \rightarrow \{0, 1\}$ such that $\alpha \models F$.

This problem was the first to be proven NP-complete by Cook [1] in 1971 and independently by Levin [2] in 1973. As a result, finding efficient algorithms for the SAT problem quickly became one of the most important and influential challenges in computer science. Modern Conflict-Driven Clause Learning (CDCL) solvers are capable of handling formulas with millions of variables and clauses. As a result, SAT solving plays a central role in hardware and software verification, automated reasoning, planning, scheduling, model checking, and cryptanalysis.

State-of-the-art solvers such as CaDiCal, Glucose, and Kissat demonstrate how theoretical advances combined with low level optimizations can transform an intractable worst-case problem into a powerful practical tool. These solvers typically focus on the decision problem: determining whether at least one satisfying assignment exists. If the formula is satisfiable, the solver returns a single model.

However, many applications require more than just one solution. In areas such as configuration, diagnosis, quantitative reasoning, explanation generation, and formal verification, it is often necessary to enumerate all satisfying assignments or a specific subset of them. This task, known as model enumeration (or All-SAT), generalizes the classical SAT problem from computing a single satisfying assignment to computing the whole set $\{\alpha : V \rightarrow \{0, 1\} \mid \alpha \models F\}$. While conceptually straightforward, model enumeration introduces significant algorithmic challenges. The number of satisfying assignments may be exponential in the number of variables, and naive approaches can quickly become infeasible.

A common technique for model enumeration is the iterative use of a SAT solver combined with blocking clauses. After each model is found, a clause composed of the negation of every literal in that model is added to the CNF formula, ensuring that the solver does not rediscover the same assignment. With the introduction of standardized interfaces such as IPASIR, SAT solvers can be seamlessly integrated into larger systems without the need to maintain the solver itself. More recently, IPASIR-UP was introduced, expanding IPASIR with User Propagators. This allows adding clauses without restarting the search. Using IPASIR-UP with Blocking Clauses yields a very clean model enumerator, that benefits from the underlying solver CaDiCal while CaDiCal itself remains unchanged. Although effective, this approach has several drawbacks. The repeated addition of blocking clauses increases the size of the formula, thereby degrading solver performance by affecting propagation efficiency and memory consumption. As the number of models grows, these effects scale up, eventually making blocking clauses infeasible.

In 2024 [3] presented TabularAllSat: A Sat solver that is especially designed

for disjoint projected enumeration for SAT and SMT without blocking clauses. TabularAllSat outperforms all other state-of-the-art solvers. This was achieved by "combining the best of current All- SAT state-of-the-art literature": (i) Conflict Driven Clause Learning short CDCL, (ii) chronological backtracking to avoid blocking clauses and (iii) implicant shrinking to minimize models.

Besides adding clauses iteratively, IPASIR-UP also allows to interfere with decision making, backtracking and the found models. This raises the question whether it is possible to use the techniques from TabularAllSat to create model enumerator through IPASIR-UP without blocking clauses.

This thesis introduces a model enumerator which is almost entirely implemented through the IPASIR-UP interface and does not need any blocking clauses. "Almost" because there are two minor additions that need to be added to IPASIR-UP: One is already added to the newest version of CaDiCaL's IPASIR-UP interface and the other is fixing a bug introduced by the former. As IPASIR-UP is an interface for CaDiCaL and CaDiCaL being a CDCL-based solver, technique (i) from TabularAllSat is naturally covered. Also technique (ii) chronological backtracking is fully possible through the IPASIR-UP interface and thus also used in this model enumerator. Using (iii) the implicant shrinking algorithm proposed by TabularAllSat requires access to the watch-lists of literals. This is not supported by the IPASIR-UP interface. However, it is still implemented in the model enumerator and section ... discusses which additional functionality is needed and how it could be added to the IPASIR-UP interface.

The model enumerator supports a fixed variable order. However, per default CaDiCaL decides. It also can be used as a simple model counter, which makes it possible to count models for formulas with exponential many models while keeping memory consumption very little. Furthermore, as IPASIR-UP also introduces the possibility to propagate literals. It is also possible to backtrack through adding reason clauses if necessary.

2 IPASIR-UP

TODO (where belongs force_backtrack) The IPASIR interface standardizes incremental interaction with CDCL-based SAT solvers by defining a minimal set of functions through which the user can call the solver and use it in bigger contexts without interacting with the solver directly and thus, making the program Independent of the solver. A solver instance is created and subsequently manipulated only via functions such as `ipasir_add` to add literals to the current clause, terminated by 0. `ipasir_assume` to add temporary assumption literals for the next call to `ipasir_solve`, and `ipasir_solve` to start or resume the search. After `ipasir_solve` returns SAT or UNSAT, the user may query models with `ipasir_val`

or inspect a failed core via `ipasir_failed`. The interface deliberately abstracts from internal solver mechanisms: There is no access to decision levels, propagation events, learned clauses, or backtracking. Although learned clauses are preserved across calls in incremental mode, the user cannot intervene during the search itself. Consequently, model enumeration requires blocking clauses: After finding a model, add its negation as a new clause to the cnf (via `ipasir_add`) and restart the solver. If the solver returns UNSAT, all models are found. However, this approach is sub-optimal as each new blocking clause increases the cnf. This makes it infeasible for big CNFs with many models. Model enumeration without blocking clauses through IPASIR is impossible since the solver terminates immediately upon finding a satisfying assignment.

To overcome these limitations, Katalin Fazekas et al. introduced IPASIR-UP in 2023 as an extension of IPASIR that introduces **user propagators** (UP). IPASIR-UP augments the classical interface with a bidirectional communication layer between the solver’s CDCL loop and user-defined callbacks. Instead of restricting interaction to clause addition and top-level solve calls, IPASIR-UP enables controlled intervention during propagation, decision making, backtracking, and model detection.

- Model validation `bool cb_check_found_model(const tmodel &model)` This function is called whenever the solver derives a total satisfying assignment. If the user propagator returns `true`, the solver reports SAT as usual. If it returns `false`, the model is rejected and the solver continues the search without restarting. Crucially, the internal trail, learned clauses, and heuristic state remain intact.
- External clause injection `bool cb_has_external_clause(bool &is_forgettable)` and `int cb_add_external_clause_lit()` These functions allow the user propagator to provide clauses to the solver right after `cb_check_found_model` was called. When `cb_has_external_clause` returns `true`, the solver repeatedly calls `cb_add_external_clause_lit` to obtain the literals of the clause (terminated by 0).
- Propagation and assignment notifications `void notify_assignment(const ivec &list)`, `void notify_new_decision_level()` and `void notify_backtrack(size_t new_level)` These notification callbacks inform the user propagator about changes to the assignment trail. `notify_assignment` communicates newly assigned literals, `notify_new_decision_level` signals that a decision was made and thus the decision level increased, and `notify_backtrack` reports to which level the solver backtracked.
- Decision making `int cb_decide()` This callback is invoked when the solver requests the next decision literal. If it returns a non-zero literal, the solver

uses it as the next decision. If 0 is returned, the solver makes its own decision, based on its internal decision heuristic.

- User propagation `int cb_propagate()` and `cb_add_reason_clause_lit cb_propagate` enables the user propagator to return implied literals under the current partial assignment. If a conflict is detected in which the propagated literal was involved, the solver asks for a reason clause through repeatedly calling `cb_add_reason_clause_lit` (terminated by 0). This enables steering through the decision space while adding the reason clauses only if really necessary.

Once a user defined an user propagator which implements all of the methods above, it can simply be connected and disconnected to the solver by calling `void connect_external_propagator (ExternalPropagator* propagator)` and `void disconnect_external_propagator (ExternalPropagator* propagator)`. The notification callback functions are only called for variables marked as `observed`. The user can mark and unmark variables through the functions `void add_observed_var (int var)` and `void remove_observed_var (int var)`.

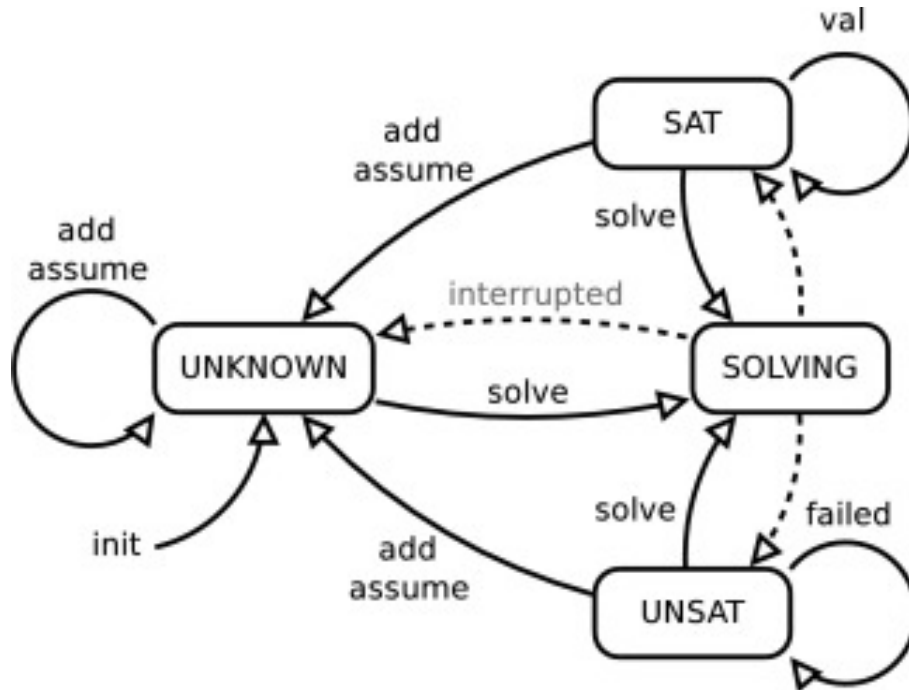


Figure 1: The four possible states of SAT solvers according to the IPASIR interface (see [?]).

Figure 1

```

1 bool cb_check_found_model(const tmodel &model) override {
2     return true; }
3
4 bool cb_has_external_clause(bool &is_forgettable) override {
5     return false; }
6
7 int cb_add_external_clause_lit() override { return 0; }

```

```

6
7 void notify_assignment(const ivec &list) override {}
8
9 void notify_new_decision_level() override {}
10
11 void notify_backtrack(size_t new_level) override {}
12
13 int cb_decide() override { return 0; }
14
15 int cb_propagate() override { return 0; }
16
17 int cb_add_reason_clause_lit(int propagated_lit) override {
    return 0; }

```

3 Deciding

The order in which decisions are made can heavily impact the performance of sat solvers. The right decision can lead the solver into early conflicts, pruning big parts of the search space completely or skipping long nested parts. While in model enumeration the later benefit is lost as all models must be found, it still profits from early conflicts.

In order to search systematically through the whole search tree, a fixed variable order is the easiest. On each `cb_decide` call, the next undecided variable is returned. Thus, assigning this variable positiv. Backtracking also becomes trivial, always backtrack to the last positive decision. However, this constraint can be weakened while still preserving a systematically order. First, the phase of a decided variable can be chosen freely, while keeping backtracking still simple enough. Instead of backtracking to the last positive decision, simply backtrack to the last decision that has a decision count of 1. Meaning it has been decided exactly once. Second, the order in which variables are decided can also be chosen complete free, as long as after a backtracking the decision is just flipped. This actually becomes non trivial as discussed in the section Chronological Backtracking. However, enables the solver to profit from dynamic decision heuristics such as Variable State Independent Decaying Sum (VISIDS) which is known to improve solver performance significantly and thus is a commonly used decision heuristic in modern sat solvers. The performance difference between fixed and dynamic decision orders can be seen in ...

4 Chronological Backtracking

Chronological backtracking can be set as an option in CaDiCaL. Thus, backtracking as a result of running into a conflict is handled by the solver itself. However, if the formula has at least one model backtracking becomes non trivial, as the path to this model must be excluded from future exploration. This can be achieved through two different methods: (i) Undo the last decision and decide its negation. (ii) When a model is found, undo the last decision and propagate its negation. While the later does not require book keeping of the decisions themselves, it may require a reason clause for that propagation if it is involved in an other backtracking. Hence introducing, even if smaller and not always, blocking clauses. Thus this thesis focuses on the former method and fully avoiding adding clauses. Still, the first method is available as a command line option (`-reason`) and also part of the benchmarks in section X.

4.1 Allowing `force_backtrack` in `cb_check_found_model`

IPASIR-UP does not allow invoking `force_backtrack` only in `cb_decide` and not in `cb_check_found_model`. However, as backtracking is needed in both cases: First, when a model was found and the user propagator returns `false`. The solver would terminates instantly as no other clauses were added and thus, the found model is still valid. Second, as describe above, after backtracking the solver may rediscover old paths. This is directly detected by `notify_assignment`. Normally `cb_decide` is called after propagation and can handle that, as calling `force_backtrack` is allowed in `cb_decide`. However, if the solver propagates all remaining variables, `cb_check_found_models` is called and it must be handled there.

As allowing `force_backtrack` to be called inside `cb_check_found_model` does not induce any bad behavior, it was officially added to IPASIR-UP by Kataling

4.2 (i) Backtracking through book keeping

In order to know which decision was made on which decision level, two information are relevant: (i) the phase of the decided variable and (ii) how often this variable has been decided. A count of 1 meaning, this decision has been made exactly once. A count of two indicating that this decision already has been flipped. Thus, finding the right decision level to backtrack to, simply becomes finding the highest decision level with a variable count of 1.

4.2.1 Backtracking after a model was found

After a model is found, either there is no decision with a decision count 1 and thus, all paths have been discovered. Then, the solver can terminate. Otherwise, there

exists a decision level with a decision count of 1 that the solver can backtrack to. Let that level be 1. Then the solver must backtrack to the decision level $1 - 1$. However, before `cb_decide` is called and the decision can be flipped, the solver propagates. This can lead to four different outcomes: (i) The solver only propagates variables that are different to the decision or none at all. In that case in `cd_decide` the decision can be flipped and returned. (ii) The solver propagates the negation of the decision itself. Then the solver can chose the next decision itself, `cb_decide` returns 0. (iii) While propagating the solver runs into a conflict and backtracks again. This case is equivalent to a normal backtrack. (iv) Due to propagation of literals the old decision itself is implied again. The solver starts to rediscover the old path. This is detected directly in `notify_assignment` but can be handled the earliest when either `cb_decide` or `cb_check_found_model` is called. Then the solver is forced to backtrack further.

4.2.2 Conflict induced backtracking

If a conflict occurs the solver automatically backtracks one decision level. However, as the solver does this on its own, it backtracks to the last decision, no matter what decision count it has. If the decision count is 1, the same four outcomes as above can occur. However, if the decision count is 2 the solver wants to flip the decision again. From this moment on, everything the solver does, must be undone again. Again this is detected and handled by the user propagator. Unfortunately, this can lead to an overhead as the solver may spend some time propagating.

4.3 (ii) Backtracking through propagation

An other approach is not to decide the negation of the last made decision but to propagate it. Thus, if the solver backtrack again, it simply backtracks over it, as it is not a decision anymore. However, if the propagated literal is inspected while conflict analysis, the solver needs a reason clause for it. The reason clause for this literal is just the negation of all decisions made on a lower level and thus can be constructed on the fly and does not need to be stored. This fits perfect with the design of IPASIR-UP as it asks for reason clauses only if necessary. Thus, adding reason clauses has two advantages to adding blocking clauses: (i) the size can be smaller as it only contains decisions on a lower level. (ii) they are only added if necessary.

5 Implicant Shrinking

...

6 Extending IPASIR-UP

There are three different proposals to IPASIR-UP: (i) Additional features that are needed in order to make model enumeration using IPASIR-UP possible. (ii) Additional features to make the optimization implicant shrinking possible and (iii) Additional features to avoid duplications and thus make model enumeration faster using IPASIR-UP.

6.1 Making IPASIR-UP the only interface needed

There are two minor changes that need to be done. First, allowing `force_backtrack` during model checking in `cb_check_found_model`. This is already added to the newest version of CaDiCaL's IPASIR-UP interface. Second, in `ask_decision` when the solver calls the user propagator. If the user propagator backtracks during `cb_decide` the solver ignores the decision made. Thus `cb_decide` must be called again. This can be achieved by tail recursively calling `ask_decision`.

6.2 Implicant Shrinking

The implicant shrinking algorithm described in section X needs access to the all watch lists of a given literal. However, the implicant shrinking algorithm uses them read only. Thus, exposing a read only reference to them is unproblematic and could be integrated into IPASIR-UP without any changes to CaDiCaL itself. Here it is important to note that literals can be named differently external and internal. However, there already exist functions to translate from and to and thus the internal representation can be completely hidden through the IPASIR-UP interface.

6.3 Optimization

Although the IPASIR-UP interface provides enough information to build an external copy of variable assignments and stack state, maintaining them takes in average XX percent of the total running time of this model enumerator. Allowing read only access to CaDiCaL's variable assignment and stack would make all external representation redundant.

7 Results

...

8 Conclusion

...

References

- [1] Stephen A. Cook. The complexity of theorem-proving procedures. *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pages 151–158, 1971.
- [2] Leonid A. Levin. Universal search problems. *Problems of Information Transmission*, 9(3):265–266, 1973.
- [3] Giuseppe Spallitta, Roberto Sebastiani, and Armin Biere. Disjoint partial enumeration without blocking clauses. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(8):8126–8135, Mar. 2024. doi: 10.1609/aaai.v38i8.28652. URL <https://ojs.aaai.org/index.php/AAAI/article/view/28652>.

Erklärung

Hiermit versichere ich, die vorliegende Arbeit ohne unerlaubte Hilfe und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt zu haben. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen entnommen sind, habe ich als solche kenntlich gemacht. Die eingereichte Masterarbeit wurde weder vollständig noch in wesentlichen Teilen Gegenstand eines anderen Prüfungsverfahrens. Die elektronische Version der eingereichten Masterarbeit stimmt in Inhalt und Formatierung mit den auf Papier ausgedruckten Exemplaren überein.

Freiburg im Breisgau, den 01.01.2016.